UNIVERSITÄT PADERBORN

Approximate Computing

Lab 2 Tutorial: Introduction to VHDL

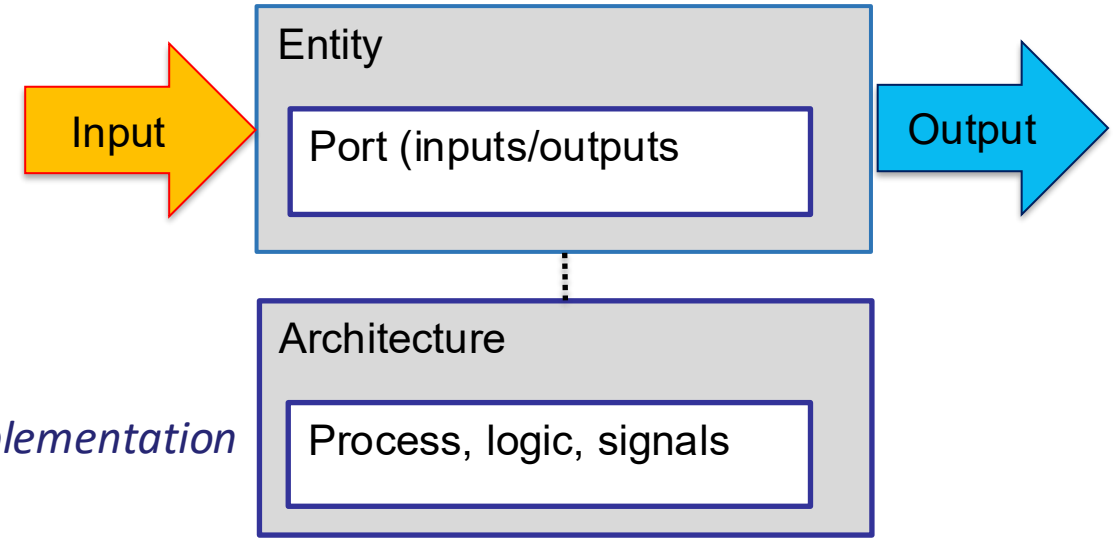
Muhammad Awais and Marco Platzner
Computer Engineering Group



What is VHDL?

- VHSIC Hardware Description Language (IEEE 1076-2008)
- Describes digital hardware — not a programming language
- Core abstraction: the design entity
  - Represents anything from a logic gate to a full system
  - Implemented at behavioral, structural, or mixed level
- Focus for this lab: RTL (Register Transfer Level) synthesis
  - Logic described as combinational blocks between registers

The entity defines the interface



The architecture defines the implementation

VHDL file structure

```
-- Library imports
library ieee;
use ieee.std_logic_1164.all;

-- Entity: defines the interface
entity fadd is
  port (
    a, b, ci : in  std_logic;   -- inputs: two bits + carry-in
    o, co    : out std_logic    -- outputs: sum + carry-out
  );
end entity fadd;

-- Architecture: defines the behavior
architecture rtl of fadd is
begin
  o  <= a xor b xor ci;
  co <= (a and b) or (ci and (a xor b));
end rtl;
```



Entity and Architecture

- Entity
  - Specifies the external interface
  - Port modes: IN, OUT (other modes exist, e.g. BUFFER)
  - Types: std_logic, std_logic_vector(N downto 0)

```
entity fadd is
  port (
    a, b, ci : in  std_logic;
    o, co    : out std_logic
  );
end fadd;
```

- Architecture
  - Implements the entity's behavior
  - Concurrent statements execute in parallel
  - Processes execute sequentially (inside process block)

```
architecture rtl of fadd is
begin
  o  <= a xor b xor ci;
  co <= (a and b) or
        (ci and (a xor b));
end rtl;
```

One entity can have multiple architectures (e.g., behavioral, structural).

A 32-bit adder example

```
-- Library imports: numeric_std needed for arithmetic on bus types
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;         -- required for unsigned()/signed()

-- Entity: 32-bit adder interface
entity adder32 is
  port (
    a, b : in  std_logic_vector(31 downto 0);
    y    : out std_logic_vector(31 downto 0)
  );
end entity adder32;

-- Architecture: behavior
architecture rtl of adder32 is
begin
   y <= std_logic_vector(unsigned(a) + unsigned(b));
end rtl;
```

Key data types: std_logic and std_logic_vector

- Use ieee.numeric_std when performing arithmetic operations
- std_logic:
  - single bit → '0', '1', 'Z' (high-Z), 'U' (uninitialized), 'X' (unknown)
- std_logic_vector(N downto 0):
  - N+1-bit bus (most common)
- For arithmetic, cast with unsigned() or signed() from numeric_std

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
-- Signal declarations (before 'begin')
-- Signal declarations (before 'begin')
signal data_bus : std_logic_vector(7 downto 0);  -- 8-bit bus
signal msb      : std_logic;
signal low      : std_logic_vector(3 downto 0);
```

```
-- Concurrent assignments (after 'begin')
msb <= data_bus(7);            -- single bit slice
low <= data_bus(3 downto 0);   -- multi-bit slice
```

Signals vs. variables

- Signals
  - Declared in architecture (outside process)
  - Assignment: s <= value
  - Update takes effect after process completes
  - Visible to the whole architecture
- Variables
  - Declared inside a process
  - Assignment: v := value
  - Update takes effect immediately
  - Local to the process only

```
-- Signal:
process(a, b)
begin
  s <= a and b;  -- scheduled
  y <= s;        -- reads OLD value of s!
end process;
```

```
-- Variable:
process(a, b)
  variable v : std_logic;
begin
  v := a and b;  -- immediate
  y <= v;        -- reads NEW value!
end process;
```

⚠ *Both signals and variables infer latches if not assigned on all branches variables make this harder to detect.*

Combinational logic

- To avoid inference of a latch
  - All inputs must appear in the sensitivity list
  - Every output signal must be assigned in all branches (if/elsif/else)
- A combinational process re-evaluates whenever any input changes

```
-- Correct combinational MUX (all inputs in sensitivity list)
entity mux32 is
  port (
    a, b : in  std_logic_vector(31 downto 0);
    sel  : in  std_logic;
    y    : out std_logic_vector(31 downto 0)
  );
end mux32;
architecture rtl of mux32 is
begin
  process(a, b, sel)       -- all inputs listed
  begin
    if sel = '1' then
      y <= a;
    else
      y <= b;            -- y assigned in BOTH branches: no latch
    end if;
  end process;
end rtl;
```

- Sensitivity list for sequential logic: clock and async reset only

```
-- 32-bit pipeline register with synchronous reset
entity pipe_reg is
  port (
    clk   : in  std_logic;
    rst   : in  std_logic;
    d_in  : in  std_logic_vector(31 downto 0);
    d_out : out std_logic_vector(31 downto 0)
  );
end pipe_reg;
architecture rtl of pipe_reg is
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        d_out <= (others => '0');   -- reset all bits to 0
      else
        d_out <= d_in;              -- register the input
      end if;
    end if;
  end process;
end rtl;
```

For loops

- Loop variable (i) is implicitly declared
- Range is inclusive: 1 to 9 iterates 1, 2, ..., 9
- Synthesizer unrolls the loop
  - this generates 9 registers in hardware, not a software loop

```
-- 10-stage shift register using a for loop (entity declaration omitted for brevity)
architecture rtl of shift_reg is
  type reg_array is array(0 to 9) of std_logic_vector(31 downto 0);
  signal stage : reg_array;
begin
  process(clk)
  begin
    if rising_edge(clk) then
      stage(0) <= d_in;          -- load input into first stage
      for i in 1 to 9 loop      -- loop variable i: no declaration needed
        stage(i) <= stage(i-1); -- each stage captures previous
      end loop;
    end if;
  end process;
  d_out <= stage(9);            -- output from last stage
end rtl;
```

Generate statement

- Structural description instantiates existing components by name using port map
- Generate statements replicate component instances
  - equivalent to a for loop for hardware instantiation

```
-- Declare the component
component fadd is
  port (
    a, b, ci : in  std_logic;
    o, co    : out std_logic
  );
end component;
-- Signals for internal carry chain
signal carry : std_logic_vector(32 downto 0);
```

Component name and ports must match the entity exactly (any mismatch causes U/X in simulation)

```
carry(0) <= '0';
-- Generate 32 full adder instances
gen_adder : for i in 0 to 31 generate
  stage_i : fadd port map (
    a  => a(i),
    b  => b(i),
    ci => carry(i),
    o  => o(i),
    co => carry(i+1)
  );
end generate;
-- carry(32) is the overflow bit
co <= carry(32);
```

Control flow: if/elsif/else and case

```
process(opcode, a, b)
begin
 if opcode = "00" then
  y <= a and b;
 elsif opcode = "01" then
  y <= a or b;
 elsif opcode = "10" then
  y <= a xor b;
 else
  y <= (others => '0’);
 end if;
end process;
```

```
process(opcode, a, b)
begin
  case opcode is
    when "00" =>
      y <= a and b;
    when "01" =>
      y <= a or b;
    when "10" =>
      y <= a xor b;
    when others =>
      y <= (others => '0');
  end case;
end process;
```

- when others is mandatory in case — covers all unspecified values
- Both if and case must cover all input combinations to avoid latches in synthesis

VHDL is NOT software!

- Concurrent execution
  - All statements outside a process execute in parallel.
- Not everything is synthesizable
  - after 10 ns is a simulation-only directive. Synthesis tools ignore it.
- A process is a hardware block
  - A process with a clock sensitivity produces a register.
  - A process without a clock produces combinational logic.
- Unassigned signals lead to latches
  - If a signal has no assignment on some path through your process, the synthesizer infers a latch. This is almost always unintended.

Testbench

- A testbench has no ports and is not synthesized
- Instantiates the Design Under Test (DUT) and drives its inputs
- Use assert to check outputs and report failures automatically

```
entity fadd_tb is end fadd_tb;   -- no ports
architecture sim of fadd_tb is
  component fadd port(a,b,ci: in std_logic; o,co: out std_logic); end component;
  signal a,b,ci,o,co : std_logic;
begin
  -- Instantiate DUT
  dut: fadd port map(a=>a, b=>b, ci=>ci, o=>o, co=>co);
  -- Stimulus process (no sensitivity list = runs once)
  stim: process
  begin
    a<='0'; b<='0'; ci<='0'; wait for 10 ns;
    assert (o='0' and co='0') report "Test 0+0+0 FAILED" severity error;
    a<='1'; b<='1'; ci<='0'; wait for 10 ns;
    assert (o='0' and co='1') report "Test 1+1+0 FAILED" severity error;
    wait;   -- stop simulation
  end process;
end sim;
```

Good practices

Incomplete sensitivity list → latch

```
-- BAD: 'b' missing from sensitivity
list
process(a)
begin
  y <= a and b;    -- latch on b!
end process;
```



```
-- GOOD
process(a, b)
begin
  y <= a and b;
end process;
```

Missing else branch → latch

```
-- BAD: y holds value when sel='0' →
latch
process(sel, a)
begin
  if sel='1' then
    y <= a;
  end if;  -- no else!
end process;
```



```
-- GOOD: always assign y
process(sel, a, b)
begin
  if sel='1' then y <= a;
  else            y <= b;
  end if;
end process;
```

Integer arithmetic without cast → error

```
-- BAD: mixing types
signal a : std_logic_vector(7 downto 0);
signal b : std_logic_vector(7 downto 0);
signal y : std_logic_vector(7 downto 0);
y <= a + b;  -- compile error!
```



```
-- GOOD: explicit cast
use ieee.numeric_std.all;
y <= std_logic_vector(
  unsigned(a) + unsigned(b)
);
```

Summary

- VHDL
  - Library / entity / architecture structure
  - std_logic and std_logic_vector types
  - numeric_std for arithmetic (unsigned/signed)
  - Combinational process → full sensitivity list
  - Sequential process → rising_edge(clk)
  - For loops
  - if/elsif/else and case with when others
  - Signals (<=) vs. variables (:=)
  - Testbench

- Lab Task (#2)
  - Implement 10-stage FIR filter in VHDL / Verilog
  - Implement approximate adders and multipliers (truncated, 1–8 bits)
  - Pre-compute component areas from DC synthesis
  - Run beam search DSE (Python/C++) to explore quality/area tradeoff
  - Evaluate MSE on 10,000-sample test set
  - Synthesize top-10 candidates with Synopsys DC &
  - Report Pareto graph of area vs. quality